

YouTube → Apple Music Bridge

Run, Stop & Authentication Guide

A practical macOS guide for running the current proof-of-concept, closing every component cleanly, and refreshing YouTube Music browser authentication when the login session changes.

Project folder: ~/yt-to-apple/youtube_bridge

Python environment: ~/yt-to-apple/youtube_bridge/venv

Important: Never share browser.json, cookies, Authorization headers, OAuth tokens, or screenshots containing them. These values can grant access to your logged-in session.

1. What you are running

Component	Purpose	Terminal
Flask service (service.py)	Local bridge: state + commands	Terminal 1
auto_bridge.py	Detects YouTube → matches Apple Music → sends playback commands	Terminal 2
Brave extension	Reports YouTube playback and controls the Apple Music tab	Brave
Apple Music Web	Receives the matched track and clicks Play	Brave

2. Before starting

1. Open Brave and make sure you are logged into YouTube / YouTube Music with the account whose recommendations you want the bridge to use.
2. Open the extension manager: **brave://extensions**.
3. Make sure Developer mode is enabled and the YouTube → Apple Music extension is enabled.
4. If you changed extension files, click **Reload** on the extension.
5. Reload the already-open YouTube tab too. This is important because content scripts do not automatically update inside an old tab after an extension reload.
6. Open Apple Music Web in Brave if you want it ready in advance. The bridge can also open its target page automatically.

3. Start the Flask service

Use a dedicated Terminal window/tab for the Flask service. Leave this terminal running.

```
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
python service.py
```

You should see something similar to:

```
YouTube → Apple Music Local Service
Listening on http://localhost:5005
```

Do not close this terminal while the bridge is running.

4. Start the automatic bridge

Open a second Terminal window/tab. Keep Terminal 1 running.

```
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
python auto_bridge.py
```

When it detects a new YouTube song, you should see output similar to:

```
YouTube detected
Video ID: ...

YouTube:
Song Name – Artist

Searching Apple Music...

Apple Music:
Song Name – Artist

Clean Apple Music URL:
https://music.apple.com/in/...

Opening Apple Music...
```

At this point the extension opens Apple Music and the Apple Music content script searches the page for the matching song and clicks its Play button.

5. Normal daily workflow

1. Start **service.py** in Terminal 1.
2. Start **auto_bridge.py** in Terminal 2.
3. Open YouTube and play a song.
4. Wait for the bridge to detect it.
5. The matching Apple Music page opens and playback starts.
6. For another YouTube song, let the new video become the current playing video; the bridge will detect the new video ID.

Current scope: this workflow is the working single-track proof-of-concept. Recommendation-to-queue synchronization is a future phase.

6. How to stop everything safely

Stop the two Python processes first. In each Python terminal, press:

```
ctrl + c
```

Do this in Terminal 2 (**auto_bridge.py**) and then Terminal 1 (**service.py**).

You do not need to deactivate the virtual environment before closing Terminal. If you want to leave it manually:

```
deactivate
```

Then you can close the Terminal windows. The Brave extension may remain loaded, but it will simply fail to contact localhost while the Flask service is stopped.

Emergency: If Ctrl+C does not stop a process, close that Terminal window. Avoid killing unrelated Python processes system-wide.

7. If you changed extension code

1. Stop `auto_bridge.py` with `Ctrl+C`.
2. You can leave `service.py` running, unless you also changed `service.py`.
3. Open **brave://extensions**.
4. Click **Reload** on the YouTube → Apple Music extension.
5. Reload the YouTube tab.
6. Reload the Apple Music tab if you changed `apple_music.js`.
7. Start `auto_bridge.py` again.

8. YouTube Music authentication: when to refresh `browser.json`

This project uses **ytmusicapi browser authentication**. The project stores the copied YouTube Music request headers in **`browser.json`**. ytmusicapi documents that these credentials remain valid while the browser session remains valid; logging out can invalidate them. `■cite■turn0search0■turn0search3■`

Refresh `browser.json` if you logged out of YouTube/YouTube Music, changed the Google account, or the bridge begins returning authentication-related errors.

9. Refresh `browser.json` — recommended method

1. Log into the correct YouTube Music account in Brave.
2. Open YouTube Music: **`music.youtube.com`**.
3. Open Brave Developer Tools with **■ + Option + I**.
4. Open the **Network** tab.
5. Use the Network filter and search for **`/browse`**.
6. If no `/browse` request appears, scroll the page or click Library to generate one.
7. Find a successful authenticated POST request to `music.youtube.com` whose status is 200.
8. Open that request and inspect its Request Headers.
9. Copy the request headers as described by ytmusicapi. The current documentation identifies the important fields as Accept, Authorization, Content-Type, X-Goog-AuthUser, x-origin, and Cookie. `■cite■turn0search0■`

Do not paste your real Cookie or Authorization values into chat. Treat them like passwords.

10. Re-create `browser.json` safely

The easiest project-specific method is to let ytmusicapi create the file:

```
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
yt musicapi browser
```

Correction: the executable is normally invoked as **ytmusicapi browser** (without a space between `yt` and `musicapi`).

```
ytmusicapi browser
```

Follow its prompt and paste the copied authentication headers. ytmusicapi's official setup command writes browser.json when a filepath is supplied; the CLI setup is the documented browser-auth flow.

■cite■turn0search0■turn0search3■

11. macOS paste problem — use pbpaste

macOS Terminal has a documented input limitation for very long pasted authentication headers. ytmusicapi specifically recommends using **pbpaste** so the clipboard contents are piped into the setup command.

■cite■turn0search0■

```
pbpaste | ytmusicapi browser
```

If your installed CLI behaves differently, use the Python setup function instead:

```
python -c 'import ytmusicapi; ytmusicapi.setup(filepath="browser.json")'
```

Then follow the prompt. The official API also supports **ytmusicapi.setup(filepath=...)** for creating the browser authentication file. ■cite■turn0search3■

12. Verify the new authentication

After updating browser.json, test the same project environment:

```
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
python poc.py
```

If your poc.py still contains a placeholder VIDEO_ID, replace it with a real YouTube video ID before testing. A successful result should show a current track and recommendations rather than an authentication error.

Then run the normal bridge again:

```
python auto_bridge.py
```

13. If you logged OUT and then logged back IN

Use this exact sequence:

1. Stop auto_bridge.py with Ctrl+C.
2. Log into the desired Google/YouTube account in Brave.
3. Open music.youtube.com and confirm the correct account is active.
4. Capture fresh authenticated headers from a successful /browse request.
5. Regenerate **browser.json** with ytmusicapi.
6. Run **poc.py** to verify authentication.
7. Start **service.py**.
8. Start **auto_bridge.py**.

You do not need to change the Python code. The scripts already load **browser.json**.

14. If you changed Google/YouTube accounts

Do not reuse the old browser.json. Generate a fresh browser.json from the newly logged-in account's authenticated YouTube Music session. The authentication file represents browser-session request headers, so the file should match the account/session you intend the bridge to use. ■cite■turn0search0■

15. If authentication stops working

Check these in order:

Check	Action
1. Account	Confirm the intended account is logged into music.youtube.com.
2. Session	Generate fresh headers from a current successful /browse request.
3. browser.json	Make sure the file exists in ~/yt-to-apple/youtube_bridge.
4. Virtual environment	Run source venv/bin/activate before testing.
5. Test	Run python poc.py before starting the full bridge.
6. Extension	Reload the Brave extension and the YouTube tab after extension changes.

16. Important security rules

- Never upload browser.json to GitHub.
- Keep browser.json in .gitignore.
- Never paste Cookie or Authorization values into ChatGPT, Discord, GitHub issues, screenshots, or reports.
- If you accidentally expose them, log out/revoke the affected session and create fresh authentication headers.
- Keep the Flask bridge bound to localhost as currently configured; do not expose port 5005 to the public internet.

17. Quick-start cheat sheet

```
# Terminal 1
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
python service.py
```

```
# Terminal 2
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
python auto_bridge.py
```

Stop: press Ctrl+C in Terminal 2, then Ctrl+C in Terminal 1.

```
# Refresh authentication after logout/login
cd ~/yt-to-apple/youtube_bridge
source venv/bin/activate
pbpaste | ytmusicapi browser
```

```
# Test
python poc.py
```

```
# Then restart the bridge
python auto_bridge.py
```

18. Current project status

Working: YouTube playback detection → YouTube title/artist identification → Apple Music catalog search → clean Apple Music URL → Apple Music Web opening → automatic matching and Play-button click.

Still future work: automatic multi-song recommendation synchronization and Apple Music queue construction/updating.

Official ytmusicapi browser-auth documentation: [url](#) [Browser authentication documentation](#) [turn0search0](#)

19. File checklist

```
youtube_bridge/  
■■■ venv/  
■■■ browser.json # private authentication; DO NOT SHARE  
■■■ service.py  
■■■ auto_bridge.py  
■■■ poc.py  
■■■ music_matcher.py  
■■■ music_controller.py  
■■■ ...
```

The Brave extension is stored separately from this Python project. Keep its background/content-script files together and reload the extension after code changes.